

RESEARCH ARTICLE

WILEY

Accuracy-first and efficiency-first privacy-preserving semantic-aware ranked searches in the cloud

Qian Zhou¹  | Hua Dai^{2,3}  | Zheng Hu² | Yuanlong Liu² | Geng Yang^{2,3}

¹School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing, China

²School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, China

³Jiangsu Key Laboratory of Big Data Security & Intelligent Processing, Nanjing University of Posts and Telecommunications, Nanjing, China

Correspondence

Qian Zhou, School of Modern Posts, Nanjing University of Posts and Telecommunications, No.66, New Model Road, Nanjing, Jiangsu Province 210003, China.

Email: zhouqian@njupt.edu.cn

Funding information

National Natural Science Foundation of China, Grant/Award Numbers: 61902199, 61872197, 61972209

Abstract

Traditional term frequency-inverse document frequency model-based privacy-preserving ranked search schemes rarely consider the latent semantic meanings of documents and keywords. It is a challenge to design efficient semantic-aware ranked search (SRSE) schemes with privacy preservation. In this paper, two privacy-preserving SRSE schemes are developed for the cloud environments. The first scheme is the accuracy-first search scheme. In this scheme, the Latent Dirichlet Allocation topic model is adopted to generate the topic-based semantic information-embedded vectors for documents and queried keywords, which supports semantic-aware relevance measurement. The bisecting k -means clustering algorithm is used to build an accuracy-first filtering tree index (AFF-tree), and the AFF-tree-based search algorithm is proposed to achieve the accuracy-first ranked search. The second scheme is the efficiency-first search scheme. It performs a structure optimization on the AFF-tree, and a newly efficiency-first filtering tree index (EFF-tree) is designed. By using the EFF-tree, an anchor node-based search algorithm is designed to achieve the efficiency-first ranked search at the expense of a little decrease in search result precision. The secure inner product is used to perform privacy-preserving semantic-aware relevance measurement between

documents and queried keywords in both schemes. To analyze the security of the proposed schemes, the game stimulation-based proof is presented. Experimental results show the better performance of the proposed schemes in search time cost.

KEYWORDS

cloud computing, privacy-preserving search, ranked search, semantic-aware search

1 | INTRODUCTION

Nowadays, cloud computing is a very successful business computing model, which can realize on-demand configurable information technology resources, such as computation, storage, network, and so forth. The resources stored in the cloud can be supplied and released quickly, and cloud computing minimizes the workload of managing resources and interacting with service providers. It distributes computing tasks on a resource pool which is formed by clustering a lot of powerful servers. More and more companies outsource their data to the cloud for saving data management costs and storage facility expenditures. Although cloud service providers claim to provide security measures such as access control and firewall for their cloud services, the privacy of outsourced data is still worrying. To solve the problem of privacy leakage, the common method is to encrypt data before outsourcing. However, the availability of encrypted data decreases, which will invalidate the traditional search algorithms. Therefore, it is necessary to investigate and apply privacy-preserving search schemes in the cloud, which could promote the development of cloud computing.

Many searchable encryption schemes have been proposed in the articles.^{1–30} These schemes all adopt the term frequency-inverse document frequency (TF-IDF) model to generate vectors for documents and search keywords. To protect the privacy of statistical information in vectors, they use the secure k -nearest neighbor⁹ to encrypt vectors into ciphertext which supports secure inner product computation. The encrypted vectors are stored in the encrypted indexes which are outsourced to the cloud together with other encrypted data. When a search with multiple interested keywords is begun by a user, the trapdoor corresponding to the interested keywords is generated and then transmitted to the cloud as a search command. The cloud server performs secure search processing by using the trapdoor and the encrypted indexes. However, the TF-IDF model is a statistical model that is used to evaluate the importance of a keyword to a document set, and it does not consider the semantic association between keywords and documents and the search results could fail to meet the search intents of users. At the same time, because of the large scale of the dictionary in practical applications, the generated vectors of documents and queried keywords, based on the TF-IDF model, are usually sparse and with very high dimensionality. It makes those search schemes not effective or efficient when performing search processing.

To solve the above problems, we introduced the Latent Dirichlet Allocation (LDA) topic model³¹ in this paper. The LDA topic model is a document-topic generation model, which is a three-layer Bayesian probability model. In the LDA topic model, each document can be

represented by a set of implicit topics, and each topic is described by the probability distribution of terms related to the topic. Therefore, the LDA topic model is capable of measuring the latent semantical correlation between keywords and documents, and the topic-based semantic-aware search can be achieved.

In this paper, we proposed two searchable encryption schemes based on the LDA topic model, which are the accuracy-first and efficiency-first privacy-preserving semantic-aware multikeyword ranked search (SRSE) schemes, respectively. In the proposed schemes, the LDA topic model is used to generate the topic information-embedded vectors for documents and queried keywords, while the secure inner product is adopted to perform privacy-preserving semantic relevance measurement between the queried keywords and documents. In the first scheme, the bisecting k -means clustering algorithm is adopted to build a novel accuracy-first filtering tree index (AFF-tree). The vectors with high relevance in the latent semantic meanings store in the adjacent leaf nodes of the AFF-tree. By using the novel index, the AFF-tree-based search algorithm is presented to achieve the accuracy-first ranked search. In the second scheme, a structure optimization is performed on the AFF-tree to generate a newly efficiency-first filtering tree index (EFF-tree). On the basis of the EFF-tree, an anchor node-based ranked search algorithm is presented. The scheme improves the search efficiency but with the expense of a little search result precision. To analyze the security of the proposed schemes, the game stimulation-based proof is presented. Experimental results show the effectiveness and efficiency of the proposed schemes.

The contributions of this article are presented in the following aspects:

1. We adopt the LDA topic model and bisecting k -means clustering algorithm to design and construct a novel AFF-tree index. The vectors in the neighboring leaf nodes of the AFF-tree are in high relevance in the latent semantic meanings. On the basis of AFF-tree, we propose the AFF-tree-based search algorithm to achieve the accuracy-first semantic-aware ranked search (AF-SRSE).
2. To further improve the search efficiency, we design a newly EFF-tree index, which is generated by linking the leaf nodes of the AFF-tree in two-way. By using the EFF-tree, we propose an anchor node-based search algorithm to achieve the efficiency-first semantic-aware ranked search (EF-SRSE).
3. To analyze the security of the schemes, we present the game stimulation-based proof which shows that the proposed schemes can preserve the privacy of documents and keywords. Moreover, we perform evaluations on a real data set to measure the search result precision and search efficiency of the proposed schemes.

The organization of the remainder of this paper is as follows: Section 2 presents the related work and Section 3 gives the notations and preliminaries; Section 4 introduces the models and problem statement of this paper; Sections 5 and 6 propose the search schemes AF-SRSE and EF-SRSE, respectively; Section 7 gives the security analysis; Section 8 presents the experimental results; and the last section concludes this paper.

2 | RELATED WORK

In recent years, searchable encryption has been widely studied, many searchable encryption schemes have been proposed. These schemes include single keyword searchable encryption schemes,¹⁻⁸ multikeyword searchable encryption schemes,^{9-21,27-30,32,33} multikeyword

searchable encryption schemes with semantic extensions,^{34–40} and so forth. We discuss the related work as follows.

2.1 | Single keyword searchable encryption schemes

Song et al.¹ presented a noninteractive single keyword search scheme for the first time, which determines whether keywords exist by scanning the encrypted files and comparing them with the ciphertext keywords. Goh et al.² formally defined a secure index structure and designed a bloom filter-based search scheme. Since the bloom filter is with false positive in a probability, the cloud server could return inaccurate search results that do not have queried keywords. Curtmola et al.³ proposed the first sublinear search time schemes SSE-1 and SSE-2. The document set is associated with an encrypted inverted index, and each entry consists of the trapdoor of the queried keyword and the document identifier. Kamara et al.⁴ extended the inverted index in the scheme,³ they proposed a formalized security definition of dynamic searchable symmetric encryption (SSE) and constructed the dynamic SSE scheme. Since the above searchable encryption schemes lack a ranking mechanism, when a large number of documents contain query keywords, it takes a long time for users to select the content they want. Wang et al.⁵ proposed a single keyword ranked search scheme, which adopts the inverted index and uses the TF-IDF model to generate vectors representing documents and queried keywords. In the search process, the cloud server calculates the relevance scores between documents and queries, and then the ranked search results are determined and returned. The above search schemes all focus on the single keyword search, but they cannot solve the multikeyword search issues.

2.2 | Multikeyword searchable encryption schemes

In a multikeyword search, users can submit multiple interested keywords as a search command to request a fixed number of documents with the highest relevance. Cao et al.⁹ proposed a multikeyword ranked search scheme with privacy preservation for the first time, which adopts the secure inner product operation. On the basis of Cao et al.,⁹ Sun et al.¹⁰ presented a secure multikeyword search scheme which adopts the TF-IDF model and builds an multidimensional B-tree⁴¹ to improve search efficiency. Xia et al.¹¹ presented a secure and dynamic multikeyword ranked search scheme which designs a balancing binary tree-based index for improving the search efficiency. Chen et al.¹² and Xiangyang et al.¹³ presented efficient multikeyword ranked search schemes by adopting the idea of clustering algorithm. Hozhabr et al.¹⁴ proposed a scheme that does not require searching the entire document, while providing dynamic manipulation of the document set. Xiao et al.¹⁵ segmented the document vector and query vector before encryption, reducing the time to construct such vectors and search. Liu et al.²¹ proposed a multikeyword fuzzy search scheme which exploits the indecomposability of prime numbers to support approximate keyword matching.

In addition, among the multikeyword searchable encryption schemes, the conjunctive keyword search schemes^{27–30} and the disjunctive keyword search schemes^{32,33} are also proposed. They are different from the above ranked search schemes, where a conjunctive keyword search requests the documents that must have all of the queried keywords while a disjunctive keyword search requests the documents that have at least one of the queried keywords. They do not support the ranking function of search results.

2.3 | Multikeyword searchable encryption schemes with semantic extensions

Since the traditional searchable encryption schemes based on the TF-IDF model do not consider the semantic association between keywords and documents, this may lead to search results that do not match the search intent of users. Searchable encryption schemes with semantic extensions have been proposed to overcome the semantic absence problem. Xia et al.³⁴ extended queried keywords with synonyms and take them as the new search to perform multikeyword search. Fu et al.³⁵ expanded the dictionary by performing keyword synonym expansion to realize the multikeyword search scheme. The above two schemes are both on the basis of the TF-IDF model. However, the TF-IDF model is not a semantic model, so the solutions hardly improve the search semantic precision. Fu et al.^{36,37} used different semantic models to implement the multikeyword search. Since the adopted models require supervised learning on given data sets, they can hardly work when applying in a scenario with different data sets. Dai et al.³⁸ first proposed an LDA topic model-based enhanced semantic-aware multi-keyword ranked search scheme (LDA-EMRSE). Documents are represented by topic vectors after the LDA topic model training. Compared with the document features extracted by the TF-IDF model, topic vectors have smaller dimensions and embed the semantic features contained in documents. However, the semantic relation between documents is not considered. There are still rooms for improvement in search efficiency. To conquer the issue, we take the topic model and clustering algorithm into the new solution design and propose two semantic-aware multikeyword ranked search schemes in this paper.

3 | NOTATIONS AND PRELIMINARIES

3.1 | Notations

- T : A set of m topics, $T = \{t_1, t_2, \dots, t_m\}$.
- D : A collection of n documents, $D = \{d_1, d_2, \dots, d_n\}$, the encrypted form of which is denoted as $\tilde{D} = \{\tilde{d}_1, \tilde{d}_2, \dots, \tilde{d}_n\}$.
- W : A keyword dictionary having h keywords, $W = \{w_1, w_2, \dots, w_h\}$.
- Θ : An $n \times m$ -dimensional document-topic relevance matrix, the encrypted form of which is denoted as $\tilde{\Theta}$.
- $\mathcal{L}$: A semantic-aware clustering document list formed by the leaf nodes in the preorder traversal of a binary clustering tree (shown in Section 5.1).
- $\mathcal{I}$: A binary tree index generated from $\mathcal{L}$ and Θ , the encrypted form of which is denoted as $\tilde{\mathcal{I}}$.
- Q : A search with multiple interested keywords of a data user, $Q \subseteq W$.
- V_Q : An m -dimensional query topic vector, the encrypted form of which is denoted as $\tilde{V}_Q$.

3.2 | LDA topic model and semantic-aware search

The LDA model is an unsupervised learning algorithm proposed by Blei.³¹ It is a three-layer Bayesian probability model extended on probabilistic Latent Semantic Indexing, which is a

graphical model for topic discovery. The LDA topic model uses topics to connect documents and keywords. In the model, each document can be represented by a set of implicit topics, and each topic is described by the probability distribution of terms related to the topic. It can be used to identify potential topic information in a large-scale document set or corpus.

We use documents in D as the input of the LDA topic model in this paper, and the model outputs two matrixes, Θ and Ω . The former is the document-topic relevance matrix while the latter is the keyword-topic relevance matrix, which describes the quantized relevance between topics and documents and keywords, respectively.

- *Document-topic vector*: Each row of Θ is the topic vector of a document that stores the relevance scores between the document and the topics in T . For example, $\Theta[i]$ is the topic vector of the document d_i .
- *Keyword-topic vector*: Each row of Ω is the topic vector of a keyword that stores the relevance scores between the keyword and the topics in T . For example, $\Omega[i]$ is the topic vector of the keyword w_i .

Since topics are defined semantically or epistemologically in the LDA topic model, we can use the topic relevance score to measure the semantic similarity between a document and a set of queried keywords, by which, the SRSEs can be implemented. Given a document d_i and a search Q , the topic relevance score between d_i and Q equals the inner product between $\Theta[i]$ and V_Q , where V_Q is the query topic vector of Q . The score calculation between d_i and Q is shown in Equation (1).

$$\text{Score}(\Theta[i], V_Q) = \Theta[i] \cdot V_Q = \sum_{j=1}^m \Theta[i][j] \times V_Q[j]. \quad (1)$$

3.3 | Bisecting k -means clustering algorithm

As a kind of hierarchical clustering method, the bisecting k -means clustering algorithm is a common clustering algorithm in data mining. The basic idea is that all points are initially taken as a cluster. Then, two points in the cluster are randomly selected as the centroids and each point in the cluster is assigned to the nearest centroid in turn. After that, one cluster is divided into two subclusters. For each generated subcluster, the above clustering procedures are performed. The clustering is stopped until some conditions are satisfied, such as the number of generated clusters exceeding a fixed threshold, the number of points of each cluster being less than a fixed threshold, and so forth. The distance between any two points, p_1 and p_2 , is usually measured by the cosine distance, which is defined in Equation (2),

$$\text{sim}(p_1, p_2) = \cos\theta = \frac{\vec{p}_1 \cdot \vec{p}_2}{\|\vec{p}_1\| \cdot \|\vec{p}_2\|}, \quad (2)$$

where $\vec{p}_1$ and $\vec{p}_2$ are the vectors of p_1 and p_2 , respectively; $\|\vec{p}_1\|$ and $\|\vec{p}_2\|$ are the norms of $\vec{p}_1$ and $\vec{p}_2$, respectively.

3.4 | Hadamard product

Given two matrices $A, B \in \mathbb{R}^{m \times n}$ and $A = [a_{ij}]$, $B = [b_{ij}]$, the Hadamard product between A and B is denoted as $A \circ B$, which is shown in Equation (3),

$$A \circ B = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \cdots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \cdots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \cdots & a_{mn}b_{mn} \end{bmatrix}, \quad (3)$$

where $\circ$ is the Hadamard operator. The result of $A \circ B$ is also an $m \times n$ -dimensional matrix.

3.5 | Relevance measurement and ranked search

The relevance score between a document and a search request (i.e., a set of queried keywords) is calculated by performing the inner product of the corresponding vectors. Assuming that x and y are two n -dimensional vectors, Equation (4) shows the calculation of the relevance score between x and y .

$$\text{Score}(x, y) = x \cdot y = \sum_{i=1}^n x[i] \times y[i]. \quad (4)$$

In this paper, vectors corresponding to documents and the queried keywords are all embedded topic information, thus the inner product of two vectors is a semantic relevance score between a document and the queried keywords.

A ranked search is to obtain k documents from D which have the k highest relevance scores to a search request and are ranked according to the scores. Here, k is the number of required documents of a user and k is usually far smaller than n . For simplicity, we use the notation Q to denote a ranked search.

3.6 | Secure inner product

The secure inner product is implemented by a homomorphic matrix encryption scheme proposed in Cao et al.⁹ Given two n -dimensional vectors x and y , they are encrypted into ciphertext by Equations (5) and (6), respectively, and the encrypted vectors are named as $\tilde{x}$ and $\tilde{y}$. M is used as a private key which is a random $n \times n$ invertible matrix.

$$\tilde{x} = M^T \cdot x, \quad (5)$$

$$\tilde{y} = M^{-1} \cdot y. \quad (6)$$

The inner product between $\tilde{x}$ and $\tilde{y}$ is equal to the inner product between x and y , that is, $\tilde{x} \cdot \tilde{y} = x \cdot y$. We give the proof shown in Equation (7), which means that the inner product between two vectors can be calculated even without knowing any plaintext values of the vectors.

$$\tilde{x} \cdot \tilde{y} = (M^T x)^T (M^{-1} y) = x^T M M^{-1} y = x \cdot y. \quad (7)$$

4 | MODELS AND PROBLEM STATEMENT

4.1 | System model and threat model

The system model adopted in this paper is shown in Figure 1, which is the same as most of the related works in Section 2. It involves three different entities: the data owner (DO), the data user (DU), and the cloud server (CS).

1. DO owns the collection of documents. In our scheme, DO first applies the LDA training⁴² on documents to get document-topic vectors. Taking the vectors as the input, DO performs the bisecting k -means clustering algorithm to build an index. Then, DO encrypts documents and the index into ciphertext. Afterwards, DO outsources the encrypted documents and index to CS.
2. DU is a legal user who is authorized to perform ranked searches on documents of DO. Once a multikeyword search is started, DU generates a trapdoor of the search. Then the trapdoor is submitted to CS as the search command. After a search is accomplished and the search result is returned in ciphertext from CS to DU, he/she performs decryption on the encrypted result and gets the final result in plaintext.
3. CS is a cluster of cloud servers, which offers storage and computation services in pay-as-need. In our scheme, CS not only provide the storage for the outsourced encrypted documents and indexes, but also provide computation resource for processing searches. Once CS receives a trapdoor from DU, CS performs a search processing by using the encrypted index. After obtaining the search result which is in ciphertext, CS returns it to DU and gets ready for the next search request.

The threat model adopted in this paper is the honest-but-curious model which is also the same as many searchable encryption schemes. Specifically, CS honestly executes instructions of the predeployed schemes and returns search results. However, CS has the curiosity of the

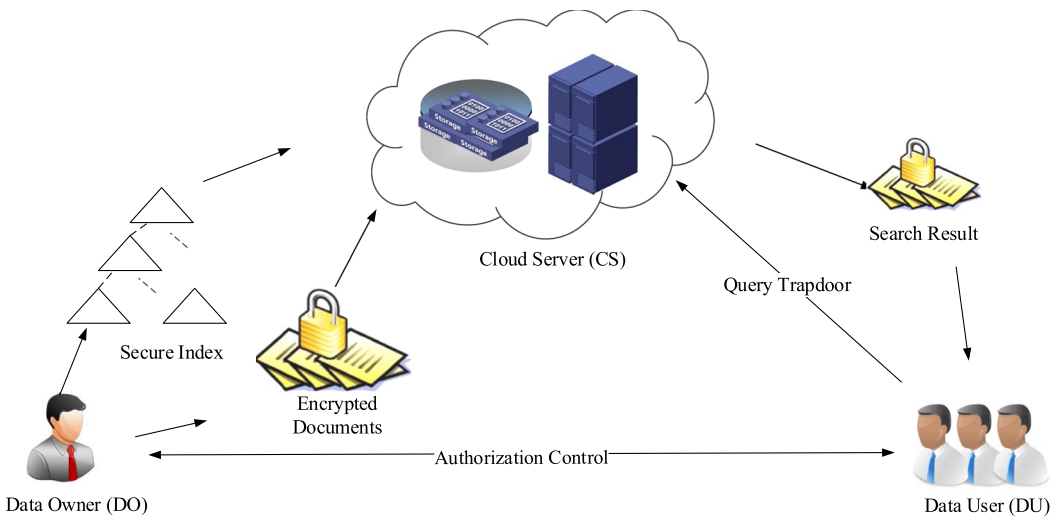


FIGURE 1 System model [Color figure can be viewed at wileyonlinelibrary.com]

private information of DO's data and DU's search intents, and CS could obtain the private information by data snooping, statistical inferring, or other methodologies.

4.2 | Security definition

We adopt the game simulation⁴³ to give the security definition of our schemes. For simplicity, Π is used to represent a searchable encryption scheme. $\mathcal{A}$ and $\mathcal{S}$ are the adversary and the simulator, respectively. Specifically, we designed two games involving $\mathcal{A}$, namely, $Real_{\mathcal{A}}^{\Pi}(\lambda)$ and $Ideal_{\mathcal{A},\mathcal{S}}^{\Pi}(\lambda)$. $Real_{\mathcal{A}}^{\Pi}(\lambda)$ is a real system that performs search processing on encrypted data, and $Ideal_{\mathcal{A},\mathcal{S}}^{\Pi}(\lambda)$ is a simulated system that $\mathcal{S}$ obtains the search result by using the leaked information of the scheme. When the queried keywords are the same, if the output results of $Real_{\mathcal{A}}^{\Pi}(\lambda)$ and $Ideal_{\mathcal{A},\mathcal{S}}^{\Pi}(\lambda)$ are indistinguishable, the scheme is $\mathcal{F}$ -adaptively secure, where $\mathcal{F}$ represents a set of leakage functions. We denote the set of leakage functions for the scheme Π as $\mathcal{F}_{\Pi}$, the definition of which will be described in the section of security analysis (Section 7). The security definition in this paper is shown as follows.

Definition 1. Given a scheme Π , if for an adversary $\mathcal{A}$, there is a simulator $\mathcal{S}$ with a set of leakage functions $\mathcal{F}_{\Pi}$ for Π such that Equation (8) holds, then Π is $\mathcal{F}$ -adaptively secure.

$$\begin{aligned} Adv_{\Pi}^{\mathcal{F}}(\mathcal{A}) &= \left| Pr \left[Real_{\mathcal{A}}^{\Pi}(\lambda) = 1 \right] - Pr \left[Ideal_{\mathcal{A},\mathcal{S}}^{\Pi}(\lambda) = 1 \right] \right| \\ &\leq negl(\lambda), \end{aligned} \quad (8)$$

where $Adv_{\Pi}^{\mathcal{F}}(\mathcal{A})$ represents the advantage that $\mathcal{A}$ can distinguish between the simulated game and the real game, and $negl(\lambda)$ is a negligible function.

4.3 | Problem description

Given a ranked search Q , the proposed search schemes should achieve the following targets:

Multikeywords ranked search: In the schemes, CS is able to return k encrypted documents for each ranked search. The result encrypted documents have the highest relevance scores to the search and are ranked according to the scores.

Search efficiency and effectiveness: The schemes are designed to achieve efficient and accurate searches by adopting novel indexes and efficient search algorithms.

Privacy preservation: The schemes are designed to protect the private information about documents, indexes, and queries from the curious CS, which are demonstrated as follows.

- *Document privacy:* The schemes should protect the plaintext of DO's documents from CS.
- *Index and query privacy:* The schemes should protect the plaintext information of the indexes and the interested keywords of DU from CS.
- *Trapdoor unlinkability¹¹:* The schemes should make CS unable to determine whether two trapdoors belong the same search requests.
- *Keyword privacy:* The schemes should make CS unable to identify any specific keywords in documents or trapdoors.

4.4 | Framework of SRSE

To introduce the SRSE schemes concisely, we present the framework shown in Figure 2, which is composed of the setup module and the search module. The former is to build the system and prepare for queries, while the latter is the search engine for users' search requests. In the first module, DO uses the algorithms *GenKey*, *LDARun*, and *BuildIndex* to create an encrypted topic information-embedding index which is outsourced to CS together with the encrypted documents. After the encrypted data are outsourced from DO to CS, the search module will be activated by CS for DU. In the second module, when a ranked search with multiple interested keywords is started, DU first uses the *GenTrapdoor* algorithm to generate a topic information-embedding trapdoor and then transmits the trapdoor to CS as the search command. CS runs the algorithm *ExSearch* to perform the privacy-preserving ranked search by using the encrypted index. When the search is accomplished, the encrypted search result is returned from CS to DU, and DU finally gets the plaintext result after decryption.

5 | ACCURACY-FIRST SEARCH SCHEME

In this section, we present an AF-SRSE scheme which adopts the LDA topic model and a clustering algorithm. We first present algorithms to generate a semantic-aware clustering document list $\mathcal{L}$ by using the bisecting k -means algorithm. Then, we design a binary tree index named AFF-tree which is constructed by taking $\mathcal{L}$ as the input. By using the AFF-tree, we give a search algorithm on plaintext which is capable of supporting accurate multikeyword ranked searches. At last, on the basis of the above methods, we propose the details of the accuracy-first search scheme.

5.1 | Semantic-aware clustering document list generation

According to the LDA topic model, each row vector $\Theta[i]$ in the document-topic relevance matrix Θ corresponds to a document d_i . Taking document-topic vectors $\{\Theta[1], \Theta[2], \dots, \Theta[n]\}$ as the input points, the bisecting k -means clustering algorithm is applied to process

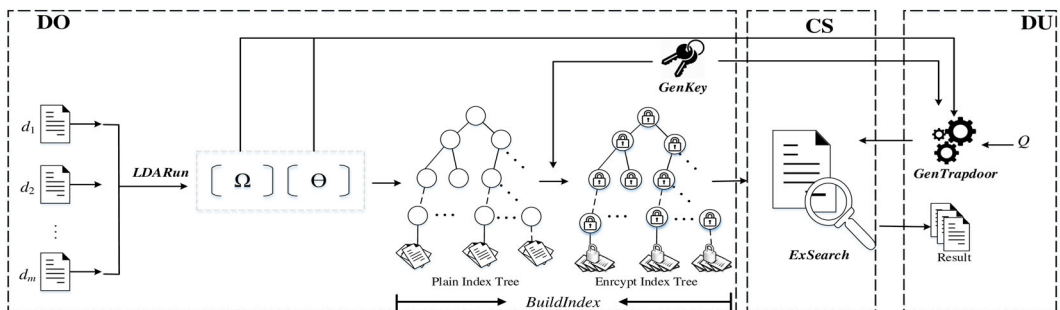


FIGURE 2 Framework of semantic-aware ranked search. CS, cloud server; DO, data owner; DU, data user. [Color figure can be viewed at wileyonlinelibrary.com]

clustering from top to down until each newly created subcluster has only one point. The clustering procedures form a semantic-aware binary clustering tree, where the root represents the initial input cluster and each leaf node is corresponding to a document. After traversing the leaf nodes of the binary tree in preorder, the list of documents corresponding to the serial traversed leaf nodes is the semantic-aware clustering document list that is named $\mathcal{L}$.

The generation of $\mathcal{L}$ is shown in Algorithm 1. It takes Θ as the input and then calls the algorithm *GenClusteringTree* (Algorithm 2) to build a binary clustering tree. After that, it calls the algorithm *TraversePreOrder* (Algorithm 3) to generate the semantic-aware clustering document list $\mathcal{L}$ and return it as the output. An example of generating $\mathcal{L}$ is shown in Figure 3.

Algorithm 1 *GenSCDList*(Θ)

Input: the document-topic vectors $\{\Theta[1], \Theta[2], \dots, \Theta[n]\}$ which are the row vectors of Θ

Output: the semantic-aware clustering document list $\mathcal{L}$

- 1: Consider $r = \{\Theta[1], \Theta[2], \dots, \Theta[n]\}$ as the initial cluster which is the root of the binary clustering tree;
 - 2: *GenClusteringTree*(r);
 - 3: Initialize the clustering document list $\mathcal{L} = \emptyset$;
 - 4: *TraversePreOrder*($r, \mathcal{L}$);
 - 5: **return** $\mathcal{L}$;
-

Algorithm 2 *GenClusteringTree*(r)

Input: r is the cluster node

Output: the root node of a clustering tree

- 1: **if** $|r| > 1$ **then**
 - 2: Apply the bisecting k -means algorithm on r to divide it into two subclusters r_1 and r_2 which are the left and right child nodes of r , respectively;
 - 3: *GenClusteringTree*(r_1);
 - 4: *GenClusteringTree*(r_2);
 - 5: **end if**
-

Algorithm 3 *TraversePreOrder*($r, \mathcal{L}$)

Input: r is the root node of a binary tree or subtree

Output: the semantic-aware clustering document list $\mathcal{L}$

- 1: **if** r is a leaf node **then**
 - 2: Append the document in r to $\mathcal{L}$;
 - 3: **else**
 - 4: *TraversePreOrder*(r .leftchild, $\mathcal{L}$);
 - 5: *TraversePreOrder*(r .rightchild, $\mathcal{L}$);
 - 6: **end if**
-

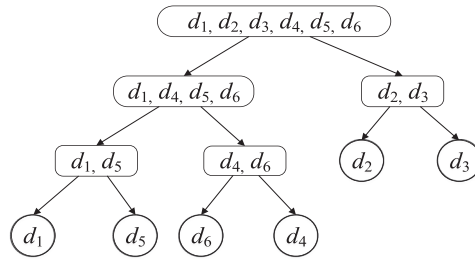


FIGURE 3 An example of generating $\mathcal{L}$. We assume that $D = \{d_1, d_2, \dots, d_6\}$ is a set of documents and $\Theta = \{\Theta[1], \Theta[2], \dots, \Theta[6]\}$ is the set of corresponding document-topic vectors. For simplicity, we use documents instead of its topic vectors to illustrate the generation of $\mathcal{L}$ in the figure. D is taken as the initial root cluster (node) and it is partitioned into two subclusters by the bisecting k -means clustering algorithm. The generated subclusters are placed as the left and right child nodes of the root. The clustering algorithm is recursively repeated on the newly generated subclusters until each of those subclusters has only one document. Finally, a binary clustering tree is constructed. Each leaf node is a subcluster that only has one document. After traversing the leaf nodes of the binary clustering tree in preorder and the semantic-aware clustering document list is generated, $\mathcal{L} = \langle d_1, d_5, d_6, d_4, d_2, d_3 \rangle$.

5.2 | AFF-tree index

The index used in AF-SRSE is a binary tree, which is named the AFF-tree. We introduce the architecture of AFF-tree and then propose the construction algorithm of it in this section.

Definition 2 (Maximized vector). Given a set of m -dimensional vectors $V = \{v_1, v_2, \dots, v_t\}$, the maximized vector of V is an m -dimension vector, which is denoted as $V^{\max}$. The value of the i th dimension of $V^{\max}$ is the maximum of values of the corresponding dimensions of every vector in V , that is,

$$V^{\max}[i] = \max\{v_j[i] | v_j \in V\}. \quad (9)$$

Definition 3 (AFF-tree). An AFF-tree is a binary tree denoted as $\mathcal{T}$, and each node u in $\mathcal{T}$ is a four-element tuple,

$$u = \langle id, lchild, rchild, vec \rangle, \quad (10)$$

where id stores an identity of a document, vec stores an m -dimensional vector, and $lchild$ and $rchild$ store the left and right child node pointers of u , respectively. The setting of the node u depends on u 's position in the tree:

- if u is a leaf node, then u corresponds to a document d_i where $u.lchild = u.rchild = \emptyset$, and $u.id$ and $u.vec$ are the identity and document-topic vector of d_i , respectively.
- if u is an internal node, then $u.lchild$ and $u.rchild$ store the pointers pointing to u 's left and right child nodes, respectively; $u.id = \emptyset$; and $u.vec$ is a filtering vector that is the maximized vector of the vectors stored in u 's left and right child nodes, that is, $u.vec = \{u.lchild.vec, u.rchild.vec\}^{\max}$.

The building of an AFF-tree is on the basis of the semantic-aware clustering document list $\mathcal{L}$ and document-topic vectors Θ . First, a list of leaf nodes is created corresponding to the documents in $\mathcal{L}$ and those leaf nodes are taken as the initial child nodes. Then, parent nodes are generated on each pair of child nodes in the bottom-up. The construction is finished when only one parent node is generated, which is the root of the tree. Details of AFF-tree construction are presented in Algorithm 4. To illustrate the architecture and construction of the AFF-tree, we give an example shown in Figure 4.

Algorithm 4 *GenIndex*($\mathcal{L}, \Theta$)

Input: $\mathcal{L}$ is the semantic-aware clustering document list and $\Theta = \{\Theta[1], \Theta[2], \dots, \Theta[n]\}$ are the document-topic vectors

Output: the AFF-tree $\mathcal{I}$

- 1: Initialize the current node list variable $CNList = \emptyset$ and the temporary node list variable $TNList = \emptyset$;
- 2: **for** each $d_i \in \mathcal{L}$ **do**
- 3: Create a newly leaf node x for d_i and set $x.id = i$, $x.lchild = x.rchild = \emptyset$ and $x.vec = \Theta[i]$;
- 4: Append x to $CNList$;
- 5: **end for**
- 6: **while** $|CNList| > 1$ **do**
- 7: **while** $|CNList| > 0$ **do**
- 8: **if** $|CNList| > 1$ **then**
- 9: Get the first two nodes x_1 and x_2 from $CNList$, and construct a new parent node x for x_1 and x_2 , where $x.id = \emptyset$, $x.lchild = x_1$, $x.rchild = x_2$ and $x.vec = \{x_1.vec, x_2.vec\}^{\max}$;
- 10: Append u to $TNList$;
- 11: **else if** $|CNList| = 1$ **then**
- 12: Fetch the only one node from $CNList$ and append it to $TNList$;
- 13: **else if** $|CNList| = 0$ **then**
- 14: Move all the nodes of $TNList$ to $CNList$ in order;
- 15: Break;
- 16: **end if**
- 17: **end while**
- 18: **end while**
- 19: The AFF-tree $\mathcal{I}$ is generated, the root of which is the only node left in $CNList$;
- 20: **return** $\mathcal{I}$;

According to Algorithm 4, we can deduce the following lemmas.

Lemma 1. *Taking $\mathcal{L}$ having n documents as the input of Algorithm 4, the height of the generated AFF-tree is $\lceil \log_2 n \rceil + 1$.*

Proof. According to Algorithm 4, the number of leaf nodes of a generated AFF-tree equals to the number of documents in $\mathcal{L}$, thus the number of leaf nodes of the generated AFF-tree is n . After the 1st, 2nd, ..., x th rounds of looping (lines 6–18) in Algorithm 4, the number of newly generated parent nodes becomes $\lceil n/2 \rceil$, $\lceil n/2^2 \rceil$, ..., $\lceil n/2^x \rceil$, respectively. When only one new parent node is generated, the loop ends and the index construction is completed. So $\lceil n/2^x \rceil = 1$ holds, and $x = \lceil \log_2 n \rceil$ is deduced. Because the height of the

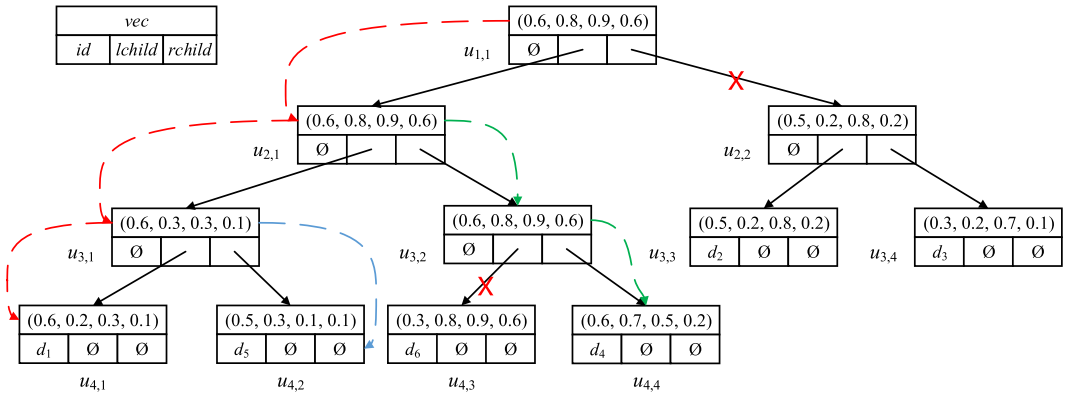


FIGURE 4 An example of AFF-tree where $\mathcal{L} = \langle d_1, d_5, d_6, d_4, d_2, d_3 \rangle$ and the number of topics is $m = 4$. This figure also shows an example of search processing by using the AFF-tree, where the query topic vector of Q is $V_Q = (0.9, 0.3, 0, 0)$ and two documents with the highest relevance scores to Q are requested. According to Algorithm 5, the search begins from the root node and reaches the first leaf node $u_{4,1}$ through the red path. The semantic relevance score between d_1 and Q is $\text{Score}(V_Q, u_{4,1}.vec) = 0.6$ and d_1 is appended to the search result R . Then, the search reaches $u_{4,2}$ through the blue path where $\text{Score}(V_Q, u_{4,2}.vec) = 0.54$, and d_5 is appended to R . At this time, the filtering threshold is updated, $\sigma = 0.54$. Then, the node $u_{4,3}$ is filtered out because $\text{Score}(V_Q, u_{4,3}.vec) = 0.51$ is smaller than σ . After that, the search reaches $u_{4,4}$ through the green path. Because $\text{Score}(V_Q, u_{4,4}.vec) = 0.75$ is larger than σ , d_4 is a candidate and it is added into R to replace d_5 that has the smaller semantic relevance score to V_Q . At the same time, the filtering threshold is updated, $\sigma = 0.6$. Afterwards, the subtree rooted by the node $u_{2,2}$ is filtered out because $\text{Score}(V_Q, u_{2,2}.vec) = 0.51$ is also smaller than σ . Finally, the result $R = \langle d_4, d_1 \rangle$ is determined. AFF-tree, accuracy-first filtering tree. [Color figure can be viewed at wileyonlinelibrary.com]

AFF-tree grows by 1 at each loop and the height is 1 at the beginning, the height of AFF-tree is $\lceil \log_2 n \rceil + 1$. Lemma 1 is proved. $\square$

Lemma 2. Given a binary tree that has n leaf nodes and the height of the tree is h , then we have

$$h \geq \lceil \log_2 n \rceil + 1. \quad (11)$$

Proof. We assume that the binary tree has N nodes. In a binary tree, there are three types of nodes which are the nodes with degrees 2, 1, and 0, respectively. The degree number of a node represents the number of child nodes that the node has. A leaf node is the node with degree 0. We assume that there are n_2 and n_1 nodes with degrees 2 and 1, respectively. Then, we have

$$N = n + n_1 + n_2. \quad (12)$$

According to the property of a binary tree, the number of nodes with degree 2 is one less than the number of nodes with degree 0 (leaf nodes) and the height of the binary tree is not lower than $\lceil \log_2(N + 1) \rceil$, that is,

$$n_2 = n - 1 \quad (13)$$

and

$$h \geq \lceil \log_2(N + 1) \rceil. \quad (14)$$

According to the above three formulas, $h \geq \lceil \log_2(2n + n_1) \rceil$ is deduced. Because of $n_1 \geq 0$, we can deduce that $h \geq \lceil \log_2(2n) \rceil = \lceil \log_2 n \rceil + 1$ holds. Lemma 2 is proved. $\square$

Lemma 3. *Given a set of leaf nodes, the AFF-tree has the lowest height in the binary trees with the same leaf nodes.*

Proof. Lemma 1 indicates that the height of the generated AFF-tree is $\lceil \log_2 n \rceil + 1$ when there are n leaf nodes. According to Lemma 2, for a binary tree that has n leaf nodes, the height of the tree is higher than or equal to $\lceil \log_2 n \rceil + 1$. Thus, we can deduce that the height of AFF-tree is the lowest in the binary trees with the same leaf nodes. Lemma 3 is proved. $\square$

The AFF-tree is an index of the AF-SRSE scheme, which makes the search processing efficiency. First, according to the structure and the construction algorithm of AFF-tree, the documents having high semantic relevance scores with each other are placed in the neighboring leaf nodes which belong to the same subtree in the AFF-tree. Thus, if one document (node) of them is accessed in search processing, then other documents (nodes) will be accessed conveniently and quickly. Second, according to Lemma 3, an AFF-tree is the lowest binary tree in those with the same number of leaf nodes. Hence, the AFF-tree is a kind of balanced binary tree which benefits the search processing in efficiency. In Section 5.3, we will introduce a plaintext search algorithm which adopts the AFF-tree.

5.3 | AFF-tree-based search algorithm

The AFF-tree-based search algorithm adopts the idea of subtree pruning to accelerate search processing. To describe the idea, we first give the following definitions and lemmas.

Definition 4. Assuming that $u \in \mathcal{I}$ is an internal node of the AFF-tree $\mathcal{I}$, the subtree rooted by u is denoted as $\mathcal{I}_u$.

Lemma 4. *Given a subtree $\mathcal{I}_u$ rooted by an internal node u in the AFF-tree and V is an m -dimensional vector storing nonnegative values, then we have*

$$\forall v \in \mathcal{I}_u \rightarrow u.vec \cdot V \geq v.vec \cdot V. \quad (15)$$

Proof. According to the definition of AFF-tree (Definition 2), the filtering vector of the internal node u is the maximized vector of the vectors stored in u 's left and right child nodes. The definition of maximized vector (Definition 1) indicates that the value of any dimension of u 's left or right child nodes' vectors are not larger than the value of the corresponding dimension of u 's vector, and so do the values of the corresponding dimension of any u 's descendant nodes' vectors. It means that for any node $v \in \mathcal{I}_u$, $u.vec[i] \geq v.vec[i]$ holds where $i \in \{1, 2, \dots, m\}$. Since values stored in a vector in this paper are not negative numbers, we can deduce $u.vec \cdot V \geq v.vec \cdot V$. Lemma 4 is proved. $\square$

Definition 5 (Filtering threshold). A filtering threshold, denoted as σ , is used for pruning subtrees in search processing with the AFF-tree.

Lemma 5. Given a filter threshold σ , a query Q whose query topic vector is V_Q , and a node u of the AFF-tree $\mathcal{I}$, if $u.vec \cdot V_Q < \sigma$ holds, then the semantic relevance scores between Q and every document in leaf nodes of the subtree $\mathcal{I}_u$ are smaller than σ , that is,

$$\forall v \in \mathcal{I}_u \wedge v.id \neq \emptyset \rightarrow v.vec \cdot V_Q < \sigma. \quad (16)$$

Proof. According to Lemma 4, for any node $v \in \mathcal{I}_u$, $u.vec \cdot V_Q \geq v.vec \cdot V_Q$ holds. Since $u.vec \cdot V_Q < \sigma$, we have $v.vec \cdot V_Q < \sigma$. It means that for each node in $\mathcal{I}_u$, the inner product result between V_Q and the vector stored in the node is smaller than σ . Therefore, the semantic relevance scores between Q and the documents in every leaf node of the subtree $\mathcal{I}_u$ are smaller than σ . Lemma 5 is proved. $\square$

The idea of the search algorithm by using the AFF-tree and the filtering threshold is described as follows. When a query Q with multiple interested keywords is performed, nodes in the AFF-tree are traversed to compute the search result. The filter threshold σ is initialized to 0 in the beginning. Once k leaf nodes (documents) have been traversed, σ is set to the minimized value of the semantic relevance scores between those documents and Q , and it becomes functional for pruning unsatisfied subtrees whose root's relevance score to Q is smaller than σ . Lemma 5 shows that the semantic relevance scores between Q and the documents in every leaf node of those unsatisfied subtrees are smaller than σ . It means that the documents in those unsatisfied subtrees have no chance to be in the search result. Thus, the unsatisfied subtrees can be directly pruned without any processing. As more leaf nodes are traversed, the filter threshold σ is increasing, and it performs better in subtree pruning which accelerates the search processing. When the traversing on the AFF-tree ends, the search result will be determined. Following the above idea, we give the details of the search algorithm *AFSearch* shown in Algorithm 5.

Algorithm 5 *AFSearch*(u, V_Q, σ, k)

Input: u is the root of an AFF-tree or a subtree in the AFF-tree, V_Q is the query topic vector of a query Q , σ is the filtering threshold that is 0 in the beginning, and k is the number of requested documents

Output: the search result R

```

1: if  $u \neq \emptyset$  then
2:   if  $\text{Score}(u.vec, V_Q) > \sigma$  then
3:     if  $u$  is an internal node then
4:       AFSearch ( $u.lchild, V_Q, \sigma, k$ );
5:       AFSearch ( $u.rchild, V_Q, \sigma, k$ );
6:     else
7:       Insert  $u$  into  $R$ ;
8:       Sort  $R$  in descending order by the semantic relevance scores of documents to the query  $Q$ ;
9:       if  $|R| > k$  then
10:        Remove the last item of  $R$ ;
11:       end if
12:     if  $|R| = k$  then
```

```

13:    $\sigma = \text{Score}(R[k].\text{vec}, V_Q);$ 
14:   end if
15: end if
16: end if
17: end if

```

According to Algorithm 5, the search processing starts from the root of the AFF-tree and the depth-first traversing is performed. The filtering threshold becomes functional for pruning unsatisfied subtrees when k leaf nodes have been accessed. The more leaf nodes are accessed, the larger the filtering threshold becomes, and the more unsatisfied subtrees will be pruned. Since only the leaf nodes in those unsatisfied subtrees are filtered out and the candidate leaf nodes are all checked and processed, the accurate search result will be determined when the AFF-tree traversing ends. Thus, the proposed AFF-tree-based search algorithm is a kind of accuracy-first search algorithm.

To illustrate the AFF-tree-based search algorithm *AFSearch*, we give an example of the search processing shown in Figure 4.

5.4 | Design of AF-SRSE

In this subsection, we give the design details of the accuracy-first semantic-aware multikey-word ranked search scheme (AF-SRSE), which provides an accurate SRSE over encrypted cloud data. According to the search framework, there are two modules in AF-SRSE: the setup module and the search module. We introduce the algorithms module by module.

1. Algorithms in setup module

$SK \leftarrow \text{GenKey}(\mathbf{1}^l(\mathbf{n}))$: The algorithm *GenKey* is performed by DO to initialize a set of private keys $SK = \{S, M_1, M_2, k_f\}$, where S is a random generated n -dimensional bit vector, M_1 and M_2 are two random $n \times n$ -dimensional invertible matrices, and k_f is a key of a selected symmetric encryption. To protect privacy from CS, SK is unknown to CS but shared by DO and DU.

$(\Theta, \Omega, V_T, V_W) \leftarrow \text{LDARun}(\mathbf{D})$: DO uses LDA to generate the matrixes Θ and Ω . Then, DO generates the prior beliefs of topic vectors and keyword vectors which are denoted as V_T and V_W , respectively. The detailed steps are given as follows.

- (a) DO extracts a set of keywords from the documents of D to generate the keyword dictionary W , and then DO performs Gibbs sampling to generate the matrixes Θ and Ω .
- (b) DO generates the topic probability vector V_T by calculating the prior belief of each topic of T existing in documents of D . We assume that the documents in D are with the identical importance. Then, the prior belief of a topic is the average value of the probabilities that each document of D belongs to the topic. The calculation of V_T is shown in Equation (19),

$$V_T = \frac{1}{n} \cdot \sum_{j=1}^n \Theta[j]. \quad (19)$$

- (c) DO generates the keyword probability vector V_W by calculating the prior belief of each keyword of W appearing in the documents of D . The calculation of V_W is shown in

Equation (20), which depends on V_T and Ω .

$$V_W = V_T \cdot \Omega^T \quad (18)$$

$\tilde{\mathcal{I}} \leftarrow \text{BuildIndex}(\Theta, SK)$: DO first build the plaintext index $\mathcal{I}$. Then, DO uses the secure keys of SK to perform encryption on the index, and the encrypted index $\tilde{\mathcal{I}}$ is generated. The procedures are given as follows:

- Taking the document-topic vectors in Θ as the input, DO uses the algorithm *GenSCDList* (shown in Algorithm 1) to generate a semantic-aware clustering document list $\mathcal{L}$ which is a key input for the latter index construction.
- Taking Θ and $\mathcal{L}$ as the input, DO uses the algorithm *GenIndex* (shown in Algorithm 4) to build a plaintext AFF-tree $\mathcal{I}$ that is used as an index for ranked searches.
- DO encrypts $\mathcal{I}$ into $\tilde{\mathcal{I}}$. First, DO uses the n -bit random vector S to split the vectors stored in every node of $\mathcal{I}$. Given a vector V stored in a node of $\mathcal{I}$, V is split following Equation (19), and two vectors $\{V', V''\}$ are generated which are both n -dimensional vectors.

$$\begin{cases} V'[i] + V''[i] = V[i], S[i] = 1, \\ V'[i] = V''[i] = V[i], S[i] = 0. \end{cases} \quad (19)$$

Then, for each split vectors $\{V', V''\}$ generated in the first step, DO uses the random invertible matrices M_1 and M_2 to encrypt them into the ciphertext $\tilde{V}$,

$$\tilde{V} = \{\tilde{V}', \tilde{V}''\} = \{M_1^T V', M_2^T V''\}. \quad (20)$$

When all nodes' vectors in $\mathcal{I}$ have been processed by split and encryption, the encrypted index $\tilde{\mathcal{I}}$ is generated.

In the end of the setup module, DO encrypts all documents of D by using the private key k_f , and the documents in the ciphertext are generated which is denoted as $\tilde{D}$. Then, $\tilde{D}$ and $\tilde{\mathcal{I}}$ are outsourced from DO to CS, and SK , Ω , V_T , and V_W are shared by DO and DU.

2. Algorithms in search module

$\tilde{V}_Q \leftarrow \text{GenTrapdoor}(Q, \Omega, V_T, V_W, SK)$: Once DU starts a search $Q = \{w_{e_1}, w_{e_2}, \dots, w_{e_p}\}$ to request the ranked k documents with top- k semantic relevance scores to Q where $w_{e_i} \in Q$ is the e_i th keyword in W , DU generates the corresponding query topic vector V_Q and the trapdoor $TD = (\tilde{V}_Q, k)$. The detailed steps are given as follows:

- The query topic vector V_Q is first calculated by Equation (21), which is an m -dimensional vector. The value of the i th dimension, $V_Q[i]$ is the probability that the topic intention of the queried keywords of Q appears in the topic $t_i \in T$.

$$V_Q = \sum_{w_{e_j} \in Q} \frac{V_T \circ \Omega[e_j]}{|Q| \cdot V_W[e_j]}. \quad (21)$$

- (b) V_Q is then spilt into two random vectors $\{V'_Q, V''_Q\}$ by using the n -bit random vector S in SK . The split rule is shown in Equation (22).

$$\begin{cases} V'_Q[i] + V''_Q[i] = V_Q, & S[i] = 0, \\ V'_Q[i] = V''_Q[i] = V_Q, & S[i] = 1. \end{cases} \quad (22)$$

- (c) By using the random invertible matrices M_1 and M_2 in SK , $V_Q = \{V'_Q, V''_Q\}$ is encrypted into $\tilde{V}_Q = \{\tilde{V}'_Q, \tilde{V}''_Q\}$ according to Equation (23).

$$\tilde{V}_Q = \{\tilde{V}'_Q, \tilde{V}''_Q\} = \{M_1^{-1}V'_Q, M_2^{-1}V''_Q\}. \quad (23)$$

Finally, the trapdoor $TD = (\tilde{V}_Q, k)$ is generated and then submitted to CS as the search command.

$R \leftarrow \text{ExSearch}(\tilde{\mathcal{I}}, \tilde{V}_Q, k)$: When CS receives the trapdoor $TD = (\tilde{V}_Q, k)$, CS executes the *AFSearch* algorithm on the encrypted AFF-tree-based index $\tilde{\mathcal{I}}$ to obtain the search result R and then returns it to DU. The relevance score between $\tilde{V}_Q$ and an encrypted vector $\tilde{V} = \{M_1^T V', M_2^T V''\}$ stored in a node of the encrypted AFF-tree $\tilde{\mathcal{I}}$ equals to the inner product between those two corresponding plaintext vectors, that is, $\text{Score}(\tilde{V}, \tilde{V}_Q) = V \cdot V_Q$. The proof is shown in formula (24). It is the reason that CS is still able to perform the search algorithm *AFSearch* to get search results even if it can only access the ciphertext of index and search commands.

$$\begin{aligned} \text{Score}(\tilde{V}, \tilde{V}_Q) &= \{M_1^T V', M_2^T V''\} \cdot \{M_1^{-1}V'_Q, M_2^{-1}V''_Q\} \\ &= (M_1^T V') \cdot (M_1^{-1}V'_Q) + (M_2^T V'') \cdot (M_2^{-1}V''_Q) \\ &= (M_1^T V')^T (M_1^{-1}V'_Q) + (M_2^T V'')^T (M_2^{-1}V''_Q) \\ &= V' \cdot V'_Q + V'' \cdot V''_Q \\ &= V \cdot V_Q. \end{aligned} \quad (24)$$

6 | EFFICIENCY-FIRST SEARCH SCHEME

In this section, we propose an EF-SRSE scheme. Compared with the above scheme AF-SRSE, EF-SRSE has higher search efficiency but at the expense of certain search precision. The main difference between EF-SRSE and AF-SRSE is the index and the search algorithm. We will first introduce the architecture and construction algorithm of the novel index used in EF-SRSE, and then, we give the index-based efficient search algorithm. At last, we give the design details of EF-SRSE.

6.1 | EFF-tree index

The index used in EF-SRSE is also a binary tree, which is named the EFF-tree. The architecture of EFF-tree is similar to the AFF-tree. The difference between EFF-tree and AFF-tree is that the leaf

nodes in EFF-tree are chained into a two-way linked list. The architecture of EFF-tree is defined as follows.

Definition 6 (EFF-tree). An EFF-tree is a binary tree denoted as $\mathcal{T}'$, and each node u in $\mathcal{T}'$ is a five-element tuple,

$$u = \langle \text{tag}, \text{id}, \text{lchild}, \text{rchild}, \text{vec} \rangle, \quad (25)$$

where id , lchild , rchild , and vec are the same to that in the AFF-tree (Definition 3), respectively. The new element tag represents the type of the node. The setting of the node u also depends on u 's position in the tree:

- If u is a leaf node, then $u.\text{tag} = 1$, and $u.\text{lchild}$ and $u.\text{rchild}$ point to u 's precursor and successor leaf nodes, respectively. The setting of $u.\text{id}$ and $u.\text{vec}$ is the same to that in Definition 3, respectively.
- If u is an internal node, then $u.\text{tag} = 0$, and the setting of $u.\text{lchild}$, $u.\text{rchild}$, $u.\text{id}$, and $u.\text{vec}$ are the same to that in Definition 3, respectively.

Following the definition of EFF-tree, we introduce the EFF-tree construction algorithm by modifying Algorithm 4. Specifically, a new step is inserted and two operations are appended to two existing steps. For simplicity, we just give the detail of the supplementary step and operations as follows.

- In Step 3 of Algorithm 4, the operation $u.\text{tag} = 1$ is appended.
- After Step 5 of Algorithm 4, a new step is inserted. The operation of the new step is that: for each leaf node $\text{CNList}[i]$ in CNList , $\text{CNList}[i].\text{lchild} = \text{CNList}[i - 1]$ and $\text{CNList}[i].\text{rchild} = \text{CNList}[i + 1]$ are set.
- In Step 9 of Algorithm 4, the operation $u.\text{tag} = 0$ is appended.

According to the definition and construction algorithm of EFF-tree, an EFF-tree has the similar shape to an AFF-tree. However, in an EFF-tree, leaf nodes storing document information are two-way linked into a list and each leaf node points to its precursor and successor leaf nodes. An example of an EFF-tree is shown in Figure 5.

6.2 | EFF-tree-based search algorithm

Definition 7 (Anchor node). For a query Q , the anchor node, denoted as $\hat{A}$, is the leaf node that has the highest semantic relevance score to Q in leaf nodes of the EFF-tree $\mathcal{T}'$, that is,

$$\forall v \in \mathcal{T}' \rightarrow \hat{A}.\text{vec} \cdot V_Q \geq v.\text{vec} \cdot V_Q, \quad (26)$$

where V_Q is the query topic vector of Q .

Definition 7 indicates that the anchor node stores the very document having the highest semantic relevance score to Q . It is the key of the EFF-tree-based search algorithm. According to the structure of EFF-tree, each leaf node chains to its precursor and successor leaf nodes by

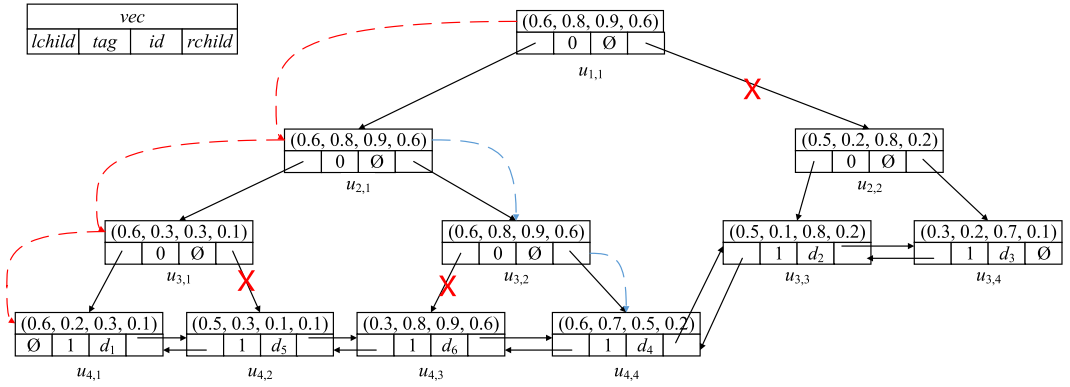


FIGURE 5 An example of EFF-tree where $\mathcal{L} = \langle d_1, d_5, d_6, d_4, d_2, d_3 \rangle$ and the number of topics is $m = 4$. This figure also shows an example of search processing by using the EFF-tree, where the query topic vector $V_Q = (0.9, 0.3, 0, 0)$, the search distance regulator $\tau = 0.5$, and top-2 most relevant documents are requested. In accordance with Algorithm 6, the search starts from the root and looks for the anchor node $\hat{A}$ by running Algorithm 7. The leaf node $u_{4,1}$ is first reached through the red dotted path. The filtering threshold σ is set to the semantic relevance score between d_1 and Q , that is, $\sigma = \text{Score}(u_{4,1}.vec, V_Q) = 0.6$, and the anchor node is currently set to $u_{4,1}$. Then, $u_{4,2}$ and $u_{4,3}$ are filtered out because $\text{Score}(u_{4,2}.vec, V_Q) = 0.54 < \sigma$ and $\text{Score}(u_{4,3}.vec, V_Q) = 0.51 < \sigma$. They have no chance to be the anchor node. Then, the search reaches the node $u_{4,4}$ through the blue dotted path. Because $\text{Score}(u_{4,4}.vec, V_Q) = 0.75 > \sigma$, the anchor node is currently turned to be $u_{4,4}$ and the filtering threshold σ is updated to 0.75. Afterwards, the subtree rooted by the node $u_{2,2}$ is filtered out because $\text{Score}(u_{2,2}.vec, V_Q) = 0.51 < \sigma$. Thus, the anchor node $\hat{A} = u_{4,4}$ is determined and added into the search result R . Since $\tau = 0.5$, the left and right neighboring leaf nodes of $\hat{A}$, $u_{4,3}$ and $u_{3,3}$, are also added into R as the candidates. Finally, the search result $R = \langle d_4, d_6 \rangle$ is determined. AFF-tree, accuracy-first filtering tree. [Color figure can be viewed at wileyonlinelibrary.com]

their left and right pointers, and thus a two-way linked list is formed. In the list, a node is probably more relevant to its neighboring nodes than those far away nodes. Thus, given a query Q , the anchor node together with its neighboring nodes in the two-way linked list probably contributes to the candidate search result.

The idea of the EFF-tree-based search algorithm is that: (1) find the anchor node $\hat{A}$; (2) take $2k\tau + 1$ leaf nodes (including the anchor node, and its forward and backward τ neighboring leaf nodes, respectively) as the candidates, where τ is the search distance regulator; (3) determine the search result in the candidates. Algorithm 6 gives the details of the EFF-tree-based search algorithm. Algorithm 7 is designed to find the anchor node, which is called in Algorithm 6.

Algorithm 6 $EFSearch(r, V_Q, k, \tau)$

Input: r is the root of an EFF-tree, V_Q is a query topic vector, k is the number of documents that DU requests, and τ is the search distance regulator;

Output: the search result R ;

(Continues)

```

1: Initialize the anchor node  $\hat{A} = \emptyset$ , two node pointers  $lp$  and  $rp$ , where  $lp = rp = \emptyset$ , the filtering threshold
    $\sigma = 0$  and a counter variable  $i = 1$ ;
2:  $\hat{A} = \text{GetAnchor}(r, V_Q, \sigma)$ ;
3: Add  $\hat{A}$  into  $R$ ;
4:  $lp = rp = \hat{A}$ ;
5: while  $i \leq \tau \cdot k$  do
6:    $lp = lp.lchild, rp = rp.rchild$ ;
7:   Add  $lp$  and  $rp$  into  $R$ ;
8:    $i = i + 1$ ;
9: end while
10: Sort  $R$  in descending order by the semantic relevance scores of documents to the query  $Q$ ;
11: Keep the first  $k$  elements in  $R$  and remove the other elements;
12: return  $R$ ;

```

Algorithm 7 $\text{GetAnchor}(u, V_Q, \sigma)$

Input: u is the root of an EFF-tree or a subtree in the EFF-tree, V_Q is a query topic vector, and σ is a filtering threshold;

Output: the anchor node $\hat{A}$;

```

1: if  $u \neq \emptyset$  then
2:   if  $\text{Score}(V_Q, u.vec) > \sigma$  then
3:     if  $u.tag = 1$  then
4:        $\sigma = \text{Score}(V_Q, u.vec)$ ;
5:        $\hat{A} = u$ ;
6:     else
7:        $\text{GetAnchor}(u.lchild, V_Q, \sigma)$ ;
8:        $\text{GetAnchor}(u.rchild, V_Q, \sigma)$ ;
9:     end if
10:  end if
11: end if

```

According to Algorithm 6, the anchor node $\hat{A}$ is first looked out through a depth-first search algorithm (Algorithm 7). Then, $2k\tau$ leaf nodes neighboring to $\hat{A}$ are obtained by using the two-way linked feature of leaf nodes, which are treated as the search result candidates together with $\hat{A}$. Compared with the algorithm $AFSearch$ in AF-SRSE, we can easily find that the traversed nodes in $EFSearch$ are obviously less than that in $AFSearch$, and thus $EFSearch$ is faster than $AFSearch$, theoretically. However, documents belonging to the search result could be stored in leaf nodes far away from the anchor node. It could cause a bit drop in search precision.

To illustrate the EFF-tree-based search algorithm, we give an example of the search processing shown in Figure 5.

6.3 | Design of EF-SRSE

In this subsection, we give the design details of the efficiency-first semantic-aware multikey-word ranked search scheme (EF-SRSE), which provides a more efficient SRSE over encrypted

cloud data. According to the search framework, EF-SRSE has the same two modules. The algorithms *GenKey*, *LDARun*, and *GenTrapdoor* in AF-SRSE can be applied for EF-SRSE except the algorithms *BuildIndex* and *ExSearch*. We modify the *BuildIndex* and *ExSearch* for EF-SRSE, which are introduced as follows.

$\mathcal{T}' \leftarrow \text{BuildIndex}(\Theta, \mathbf{SK})$: The algorithm *BuildIndex* in EF-SRSE is similar to that in AF-SRSE. There are also three steps in EF-SRSE, and the second step is different. In the second step of *BuildIndex* in EF-SRSE, DO uses the EFF-tree construction algorithm (the idea of which is shown in Section 6.1) to build an EFF-tree $\mathcal{T}'$. The other steps are the same.

$R \leftarrow \text{ExSearch}(\mathcal{T}', \tilde{V}_Q, \mathbf{k}, \tau)$: When CS receives a trapdoor of a ranked search from DU, CS executes the algorithm *EFSearch* (shown in Algorithm 6) on the encrypted EFF-tree-based index $\mathcal{T}'$ to obtain the search result R and then returns it to DU.

According to the analysis of *EFSearch* algorithm, EF-SRSE performs better in search efficiency but at the expense of certain search precision. We will evaluate the search efficiency and precision of the two proposed schemes in Section 8.

7 | SECURITY ANALYSIS

To analyze the security of the proposed schemes, we first provide a formal definition of the leakage functions in Definition 1.

Definition 8. $\mathcal{F}_{\Pi} = \{\mathcal{F}^{\text{GK}}, \mathcal{F}^{\text{Set}}, \mathcal{F}^{\text{BI}}, \mathcal{F}^{\text{GT}}, \mathcal{F}^{\text{Search}}\}$ is a set of leakage functions for the proposed schemes, which corresponds to the algorithms *GenKey*, *Setup*, *BuildIndex*, *GenTrapdoor*, and *ExSearch* in the proposed schemes, respectively. Specifically, the leakages are described as follows:

- Leak_{SP} is the search pattern leakage, which reflects the same results acquired by different trapdoors.
- Leak_{AP} is the access pattern leakage, which is the identities of the returned encrypted documents together with the document ranking information.
- Leak_{SIP} is the size pattern leakage, which is the number of encrypted documents, the length of an encrypted document vector, and the length of a trapdoor.

The leakage functions $\mathcal{F}^{\text{GK}}$, $\mathcal{F}^{\text{Set}}$, $\mathcal{F}^{\text{BI}}$, and $\mathcal{F}^{\text{GT}}$ correspond to Leak_{SIP} while the leakage function $\mathcal{F}^{\text{Search}}$ corresponds to Leak_{SP} and Leak_{AP} .

Theorem 1. The proposed schemes, AF-SRSE and EF-ESRSE, are $\mathcal{F}$ -adaptively secure with the leakage functions of $\mathcal{F}_{\Pi}$.

Proof. We adopt the game simulation-based proof method to prove the theorem. In the ideal game, $\text{Ideal}_{\mathcal{A}, \mathcal{S}}^{\Pi}(\lambda)$, $\mathcal{S}$ simulates the steps in AF-SRSE and EF-ESRSE according to the given leak functions in $\mathcal{F}_{\Pi}$. If the adversary $\mathcal{A}$ cannot distinguish between the outputs of the real game and the ideal game, we conclude that the schemes are $\mathcal{F}$ -adaptively secure with the leakage functions of $\mathcal{F}_{\Pi}$.

Game G_0 : The game G_0 is the real-world game $Real_A^\Pi(\lambda)$, thus we have

$$Pr[Real_A^\Pi(\lambda) = 1] = Pr[G_0^A = 1]. \quad (27)$$

Game G_1 : The game G_1 is the simulated-ideal game $Ideal_{A,S}^\Pi(\lambda)$, thus we have

$$Pr[Ideal_{A,S}^\Pi(\lambda) = 1] = Pr[G_1^A = 1]. \quad (28)$$

In G_1 , the simulator $\mathcal{S}$ first generates nm -dimensional random vectors $\Theta' = \{\Theta'[1], \Theta'[2], \dots, \Theta'[n]\}$ as the encrypted document-topic vectors by using the leakages $Leak_{SP}$, $Leak_{AP}$, and $Leak_{SIP}$. Next, $\mathcal{S}$ constructs the tree-based indexes on Φ' according to the index construction algorithms in the proposed schemes. For a given query Q , $\mathcal{S}$ generates an m -dimensional random vector V'_Q as the trapdoor and then stores the pair (Q, V'_Q) for reuse.

We analyze the indistinguishability between G_0 and G_1 to prove the theorem. According to the search algorithms (Algorithms 5 and 6) in the proposed schemes, the key to achieve searches over encrypted data is to generate the encrypted document-topic vectors and query topic vectors that support secure inner product operations, where the random invertible matrices M_1 and M_2 in key $\mathcal{SK}$ are used to generate encryption vectors according to Equations (20) and (23). Therefore, the advantage that $\mathcal{A}$ can distinguish G_0 and G_1 is equivalent to the advantage that $\mathcal{A}$ can crack the matrix-based encryption shown in Equation (29), which is the advantage of solving Equation (29) to obtain M_1 , M_2 , V' , and V'' with given $\widetilde{V}'$ and $\widetilde{V}''$.

$$\begin{cases} V' \times M_1 = \widetilde{V}', \\ V'' \times M_2 = \widetilde{V}''. \end{cases} \quad (29)$$

In the equations of Equation (29), M_1 and M_2 are the $m \times m$ -dimensional random invertible matrices, V' and V'' are the m -dimensional vectors, and $\widetilde{V}'$ and $\widetilde{V}''$ are the m -dimensional encrypted vectors, so the equations have $2m \times (m + 1)$ unknowns and $2m$ known values. It is computationally infeasible to solve such equations to obtain M_1 , M_2 , V' , and V'' in polynomial time. It means that the advantage of distinguishing between randomly generated numbers and matrix-based encryption results is negligible, thus it is negligible to distinguish between G_0 and G_1 , that is,

$$|Pr[G_0^A = 1] - Pr[G_1^A = 1]| \leq \text{negl}(\lambda). \quad (30)$$

According to Equations (27), (28), and (30), we derive that

$$\left| Pr[Real_A^\Pi(\lambda) = 1] - Pr[Ideal_{A,S}^\Pi(\lambda) = 1] \right| \leq \text{negl}(\lambda). \quad (31)$$

According to the security definition of the scheme in this paper (Definition 1), the proposed schemes in this paper are $\mathcal{F}$ -adaptively secure. Theorem 1 is proved. $\square$

8 | PERFORMANCE EVALUATION

In this section, we perform evaluations of the proposed schemes AF-SRSE and EF-SRSE, and present comparisons between them and the schemes TFIDF-MRSE¹¹ and LDA-EMRSE.³⁸ We implement these four schemes by using python 3.6 in Windows 10 with an Intel Core(TM) i5-9300H. The evaluation is performed from two metrics. One is the semantic precision of search result and the other is the search time cost. The data set used in the evaluation is 20 newsgroups.⁴⁴ It has 11,315 news articles which are categorized into 20 different categories. Table 1 shows the default value setting of parameters, where n is the quantity of documents in D , m is the quantity of topics in T , $|Q|$ is the quantity of queried keywords in Q , k is the quantity of requested documents of users, and τ is the search radius which is only used in EF-SRSE.

8.1 | Semantic precision evaluation

To reasonably describe the semantic precision of search result for a search scheme, the keywords of a given query are assumed to belong to the same category and the documents belonging to the different categories are weakly relevant or irrelevant in topic semantic. We utilize the precision measurement adopted in Gabryel et al.⁴⁵ to evaluate the semantic precision of four schemes. The calculation formula is shown in Equation (32).

$$Precision = \frac{TP}{TP + FP} \times 100\%.$$
 (32)

Here, TP is the quantity of documents in the result which are in the same category of the queried keywords while FP is the quantity of documents in the result which are not in the category that the queried keywords belong to.

- 1. *Semantic precision versus n*: Figure 6A shows that as n increases, the semantic precisions of AF-SRSE, EF-SRSE, and LDA-EMRSE remain stable, while the semantic precision of TFIDF-MRSE gradually decreases. The reason is that in AF-SRSE, EF-SRSE, and LDA-EMRSE, as the number of documents grows, the topics extracted by the LDA topic model can better reveal the latent semantic features of documents, which has no significant affections on the semantic precision of search results.
- 2. *Semantic precision versus k*: Figure 6B shows that as k increases, the semantic precisions of four schemes all gradually decrease. The reason is that as k increases, the documents with

TABLE 1 Default value setting of parameters

Parameter notations	n	m	$ Q $	k	τ
Default values	8000	100	3	100	1.0

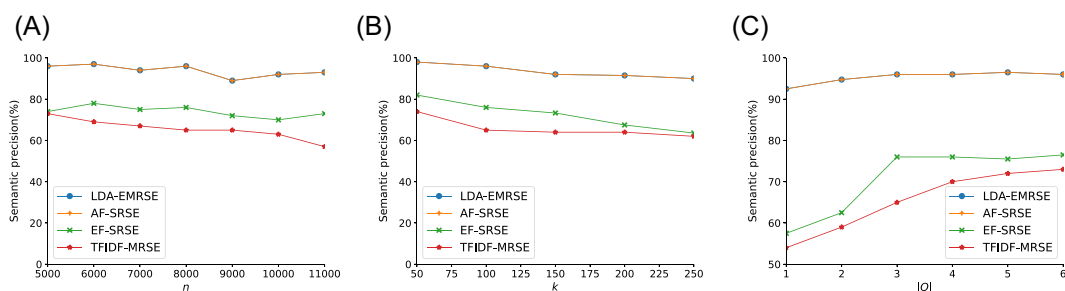


FIGURE 6 Semantic precision versus n , k , and $|Q|$. AF-SRSE, accuracy-first SRSE; EF-SRSE, efficiency-first SRSE; LDA, Latent Dirichlet Allocation; SRSE, semantic-aware ranked search [Color figure can be viewed at wileyonlinelibrary.com]

lower scores to the queried keywords are appended to the final result, and it could lead to a bit decrement in the semantic precisions of search results.

3. *Semantic precision versus $|Q|$* : Figure 6C indicates that as $|Q|$ increases, the semantic precisions of AF-SRSE, EF-SRSE, and LDA-EMRSE and TFIDF-MRSE all gradually increase and then become stable. It is because the more queried keywords exist in a multikeyword ranked search, the more semantic characteristics are implicitly described in the search, and the more accurate search result is generated. However, when the queried keywords have been able to well represent the semantic characteristics of the query, continuing to increase the number of queried keywords will have less impact on the result. At the same time, the semantic precisions of AF-SRSE and LDA-EMRSE are the same and the highest, EF-SRSE is the second and TFIDF-MRSE is the lowest.

In the above three semantic precision evaluations, Figure 6A–C all show that AF-SRSE and LDA-EMRSE are the same and the highest, EF-SRSE comes second and TFIDF-MRSE is the lowest. The reason is that in AF-SRSE and LDA-EMRSE, they both traverse the documents with higher relevance scores and they will not omit the documents that are highly relevant to the query, so the semantic precisions of them are the same. Different from AF-SRSE and LDA-EMRSE, EF-SRSE only finds the document with the highest relevance score and then stops traversing the index. It results in lower semantic precision. Because AF-SRSE, EF-SRSE, and LDA-EMRSE all use the LDA topic model to train the document-topic vectors, which better express the semantic information of documents and keywords than the TF-IDF model, their semantic precisions are thus higher than TFIDF-MRSE.

8.2 | Search time cost evaluation

We first measure the time cost of inner products and give the proportion that takes in the total time cost, and then we evaluate the search time cost when n , k , and $|Q|$ change, respectively. In Figures 7–9, there are three subfigures in each figure, respectively, which are the subfigures (A)–(C). As shown in Figures 7A, 8A, and 9A, the search time costs of AF-SRSE, EF-SRSE, and LDA-EMRSE are too lower than that of TFIDF-MRSE to clearly distinguish. We redraw the evaluation results of AF-SRSE, EF-SRSE, and LDA-EMRSE in independent figures, Figures 7B, 8B, and 9B, respectively. Additionally, Figures 7C, 8C,

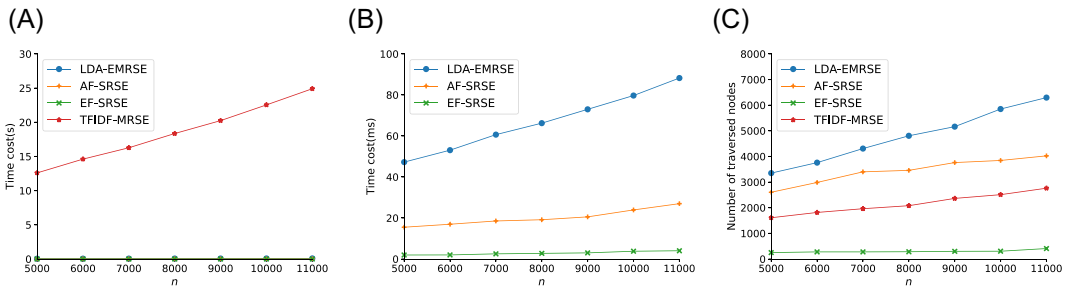


FIGURE 7 Time cost versus n . AF-SRSE, accuracy-first SRSE; EF-SRSE, efficiency-first SRSE; LDA, Latent Dirichlet Allocation; SRSE, semantic-aware ranked search. [Color figure can be viewed at wileyonlinelibrary.com]

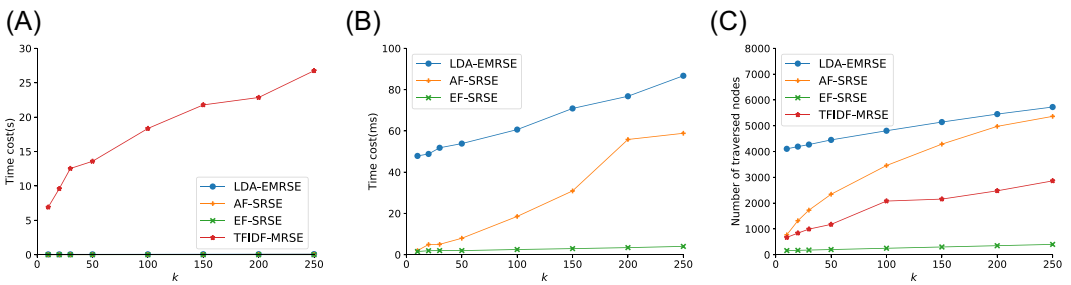


FIGURE 8 Time cost versus k . AF-SRSE, accuracy-first SRSE; EF-SRSE, efficiency-first SRSE; LDA, Latent Dirichlet Allocation; SRSE, semantic-aware ranked search. [Color figure can be viewed at wileyonlinelibrary.com]

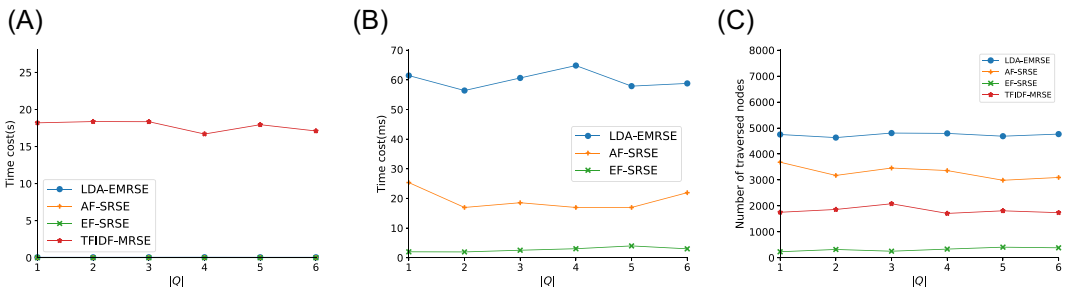


FIGURE 9 Time cost versus $|Q|$. AF-SRSE, accuracy-first SRSE; EF-SRSE, efficiency-first SRSE; LDA, Latent Dirichlet Allocation; SRSE, semantic-aware ranked search. [Color figure can be viewed at wileyonlinelibrary.com]

and 9C show the numbers of traversed nodes when processing searches in four schemes, respectively.

1. *Time cost of inner products*: Table 2 shows the proportions of time costs that the inner products take in the total time costs in four schemes, respectively. The inner product time cost is denoted as IPTC. The result indicates that the inner product operation is the main time-consuming operation in four schemes.

2. *Time cost versus n* : The results of Figure 7A,B show that as n grows, the search time costs of four schemes all grow. It is because the four schemes all have to traverse nodes

TABLE 2 Inner product time cost evaluation

	Total (ms)	IPTC (ms)	IPTC/total (%)
TFIDF-MRSE	18,345	17,684.58	96.4
LDA-EMRSE	66.24	55.97	84.5
AF-SRSE	19.15	17.1	89.3
EF-SRSE	2.76	2.44	88.3

Abbreviations: AF-SRSE, accuracy-first SRSE; EF-SRSE, efficiency-first SRSE; IPTC, inner product time cost; LDA, Latent Dirichlet Allocation; SRSE, semantic-aware ranked search.

selectively and perform an inner product operation on vectors in each traversed node to look for search results. The larger n is, the more traversed nodes are which is shown in Figure 7C, and thus the more inner product operations are performed. It results in the increase of search time cost.

3. *Time cost versus k* : Figure 8A,B shows that as k grows, the search time costs of four schemes all increase. This is because as k increases, more nodes are traversed to look for those incremental requested documents, and thus more inner product operations on vectors are performed. So the time costs all increased. Additionally, in four schemes, the growth rate of time cost in EF-SRSE is obviously smaller than that in the other schemes. The reason is that as k grows, the number of incremental traversed nodes in EF-SRSE is much less than those in TFIDF-MRSE, LDA-EMRSE, and AF-SRSE, respectively, which is shown in Figure 8C. It means that the incremental inner product operations in EF-SRSE are much less than those in other schemes, and thus the growth rate of search time cost in EF-SRSE is the smallest.

4. *Time cost versus $|Q|$* : Figure 9A,B indicates that as $|Q|$ grows, the search time costs of four schemes are stable. It means that the number of queried keywords has a rare impact on the search time cost. The reason is that no matter how many keywords are queried, they will be always converted into vectors with fixed numbers of dimensions in those four schemes, respectively. The numbers of traversed nodes in those schemes are stable and oscillate in certain ranges, respectively, which is shown in Figure 9C. Thus, the time costs of four schemes almost have nothing to do with the number of queried keywords.

In Figures 7A, 8A, and 9A, the evaluation results all show that the search time cost of TFIDF-MRSE is much higher than that of the other three schemes. The reason is that the dimension of a vector in TFIDF-MRSE equals the number of keywords in the dictionary, which is significantly higher than those in the other three schemes. It consumes a lot of time when performing inner product operations on vectors. Figures 7B, 8B, and 9B all show that in the other three schemes, the time cost of LDA-EMRSE is the highest, AF-SRSE is the second, and EF-SRSE is the lowest. The reason is that those three schemes all use the binary tree-based index to perform searches, and documents are all stored in leaf nodes of indexes, respectively. In LDA-EMRSE, documents are randomly distributed in leaf nodes. While in AF-SRSE and EF-SRSE, documents are arranged into leaf nodes according to the document clustering result, where the documents in neighboring leaf nodes are with high semantic relevance. Thus, when a search is processed, more nodes will be traversed in LDA-EMRSE than those in AF-SRSE and EF-SRSE, respectively, and hence LDA-EMRSE consumes more time. Additionally, since the index structure and search algorithm in EF-SRSE are optimized on the basis AF-SRSE, less nodes are traversed in EF-SRSE than that in AF-SRSE, and thus EF-SRSE is more efficient than AF-SRSE in search processing.

8.3 | Parameter setting on τ in EF-SRSE

We evaluate and analyze the impact of τ on search precision and time cost in EF-SRSE, respectively, because τ is only involved in the scheme EF-SRSE rather than the other three schemes. We execute three searches with the same queried keywords and requested top-50, top-100, and top-150 documents to perform the evaluation, respectively.

1. *Semantic precision versus τ* : Figure 10 shows that as τ increases from 0.5 to 1.4, the semantic precision of EF-SRSE first gradually increases and then becomes stable. The reason is that as τ grows, the scope of left-to-right exploration around the anchor node becomes wider. It means that more documents with high relevance become candidates. Therefore, the semantic precision first grows up. However, when τ continues to grow up, those incremental candidate leaf nodes become more and more far away from the anchor node. So the relevance between the document of the anchor node and the newly added candidate documents is becoming lower and lower, and the impact of τ on the semantic precision is getting smaller and smaller. It results in the stable semantic precision at last. According to the result of Figure 10, the optional value of τ is about 1.0.

2. *Time cost versus τ* : Figure 11A shows that as τ increases, the time costs of three searches all gradually grow up. The reason is that τ only affects the search radius from the anchor node.

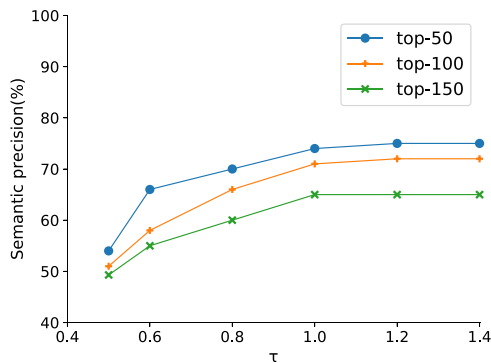


FIGURE 10 Semantic precision versus τ [Color figure can be viewed at wileyonlinelibrary.com]

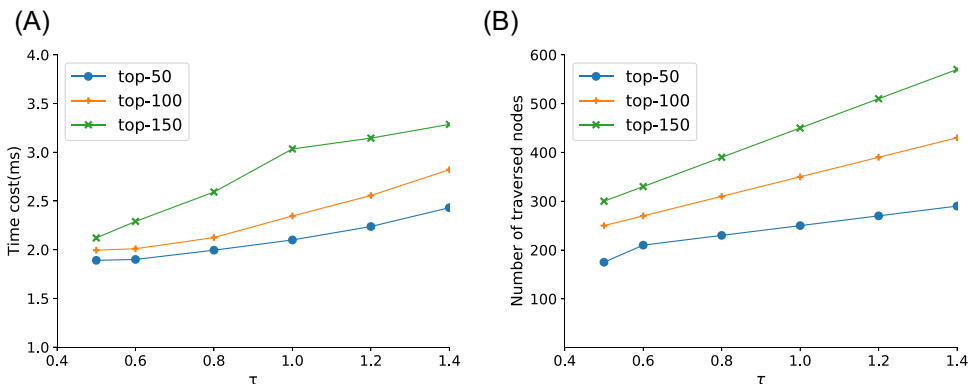


FIGURE 11 Time cost versus τ [Color figure can be viewed at wileyonlinelibrary.com]

When τ increases, the number of traversed nodes increases which is shown in Figure 11B. It means that more inner product operations are performed and thus the search time costs all increase.

9 | CONCLUSION

Since the traditional searchable encryption schemes based on the TF-IDF model do not consider the semantic association between keywords and documents, we adopt the LDA topic model to consider the user's potential search intentions. Documents are modeled as the topic-based vectors containing semantic information through the LDA topic model. To further improve the search efficiency, this paper proposes two schemes based on the LDA topic model, namely, the accuracy-first semantic-aware multikeyword ranked search scheme (AF-SRSE) and the efficiency-first semantic-aware multikeyword ranked search scheme (EF-SRSE). Both of them use clustering ideas to tightly store documents with similar semantics in the tree-based index, which greatly improve the search efficiency. The latter optimized the index of the former, and the search efficiency is further improved than the former but with the expense of a little search result precision. The game stimulation-based security analysis shows that the proposed schemes can preserve the privacy of data in search processing. The experiment results indicate the efficiency and effectiveness of the proposed scheme.

ACKNOWLEDGMENTS

This study is supported by the National Natural Science Foundation of China under Grant Nos. 61872197, 61902199, and 61972209.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Qian Zhou  <http://orcid.org/0000-0001-7888-2419>

Hua Dai  <http://orcid.org/0000-0003-2465-8977>

REFERENCES

1. Song DX, Wagner D, Perrig A. Practical techniques for searches on encrypted data. In: *Proceedings of the 2000 IEEE Symposium on Security and Privacy. S&P 2000*. IEEE; 2000:44-55. doi:10.1109/SECPRI.2000.848445
2. Goh EJ. Secure indexes. *IACR Cryptol ePrint Arch*. 2003;2003:216.
3. Curtmola R, Garay J, Kamara S, Ostrovsky R. Searchable symmetric encryption: improved definitions and efficient constructions. *J Comput Secur*. 2011;19(5):895-934.
4. Kamara S, Papamanthou C, Roeder T. *Dynamic Searchable Symmetric Encryption*. ACM; 2012:965.
5. Wang C, Cao N, Li J, Ren K, Lou W. Secure ranked keyword search over encrypted cloud data. In: *2010 IEEE 30th International Conference on Distributed Computing Systems*. IEEE; 2010:253-262. doi:10.1109/ICDCS.2010.34

6. Wang C, Cao N, Ren K, Lou W. Enabling secure and efficient ranked keyword search over outsourced cloud data. *IEEE Trans Parallel Distrib Syst*. 2011;23(8):1467-1479.
7. Chang YC, Mitzenmacher M. Privacy preserving keyword searches on remote encrypted data. In: John I, Angelos K, Moti Y, eds. *International Conference on Applied Cryptography and Network Security*. Springer; 2005:442-455.
8. Liu C, Zhu L, Chen J. Efficient searchable symmetric encryption for storing multiple source dynamic social data on cloud. *J Network Comput Appl*. 2017;86:3-14.
9. Cao N, Wang C, Li M, Ren K, Lou W. Privacy-preserving multi-keyword ranked search over encrypted cloud data. In: *IEEE INFOCOM 2011*; 2011:829-837. doi:10.1109/INFCOM.2011.5935306
10. Sun W, Wang B, Cao N, et al. Verifiable privacy-preserving multi-keyword text search in the cloud supporting similarity-based ranking. *IEEE Trans Parallel Distrib Syst*. 2014;11(25):3025-3035.
11. Xia Z, Wang X, Sun X, Wang Q. A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans Parallel Distrib Syst*. 2015;27(2):340-352.
12. Chen C, Zhu X, Shen P, et al. An efficient privacy-preserving ranked keyword search method. *IEEE Trans Parallel Distrib Syst*. 2016;27(4):951-963.
13. Zhu X, Dai H, Yi X, Yang G, Li X. MUSE: an efficient and accurate verifiable privacy-preserving multikeyword text search over encrypted cloud data. *Secur Commun Networks*. 2017;2017:1-17.
14. Hozhabr M, Asghari P, Javadi H. Dynamic secure multi-keyword ranked search over encrypted cloud data. *J Inf Secur Appl*. 2021;61(1):102902.
15. Xiao T, Han D, He J, Li KC, Mello dRF. Multi-keyword ranked search based on mapping set matching in cloud ciphertext storage system. *Connect Sci*. 2021;33(1):95-112.
16. Hua J, Liu Y, Chen H, Tian X, Jin C. Geometric range search on encrypted data with forward/backward security. *World Wide Web*. 2020;23:2185-2214.
17. Cao N, Wang C, Li M, Ren K, Lou W. Privacy-preserving multi-keyword ranked search over encrypted cloud data. *IEEE Trans Parallel Distrib Syst*. 2014;1(25):222-233.
18. Wang C, Ren K, Yu S, Urs KMR. Achieving usable and privacy-assured similarity search over outsourced cloud data. In: *2012 Proceedings of the IEEE INFOCOM*. IEEE; 2012:451-459. doi:10.1109/INFCOM.2012.6195784
19. Zhang W, Xiao S, Lin Y, Zhou T, Zhou S. Secure ranked multi-keyword search for multiple data owners in cloud computing. In: *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE; 2014:276-286.
20. Tseng CY, Lu C, Chou CF. Efficient privacy-preserving multi-keyword ranked search utilizing document replication and partition. In: *2015 12th Annual IEEE Consumer Communications and Networking Conference (CCNC)*. IEEE; 2015:671-676. doi:10.1109/CCNC.2015.7158059
21. Liu Q, Peng Y, Wu J, Wang T, Wang G. Secure multi-keyword fuzzy searches with enhanced service quality in cloud computing. *IEEE Trans Network Service Manage*. 2020;18(2):2046-2062.
22. Fu Z, Sun X, Ji S, Xie G. Towards efficient content-aware search over encrypted outsourced data in cloud. In: *IEEE INFOCOM 2016—The 35th Annual IEEE International Conference on Computer Communications*. IEEE; 2016:1-9. doi:10.1109/INFCOM.2016.7524606
23. Li J, Wang Q, Wang C, Cao N, Ren K, Lou W. Fuzzy keyword search over encrypted data in cloud computing. In: *2010 Proceedings of the IEEE INFOCOM*. IEEE; 2010:1-5. doi:10.1109/INFCOM.2010.5462196
24. Wang B, Yu S, Lou W, Hou YT. Privacy-preserving multi-keyword fuzzy search over encrypted data in the cloud. In: *IEEE INFOCOM 2014—IEEE Conference on Computer Communications*. IEEE; 2014:2112-2120. doi:10.1109/INFCOM.2014.6848153
25. Zhu X, Liu Q, Wang G. A novel verifiable and dynamic fuzzy keyword search scheme over encrypted data in cloud computing. In: *2016 IEEE Trustcom/BigDataSE/ISPA*. IEEE; 2016:845-851. doi:10.1109/TrustCom.2016.0147
26. Ge X, Yu J, Hu C, Zhang H, Hao R. Enabling efficient verifiable fuzzy keyword search over encrypted data in cloud computing. *IEEE Access*. 2018;6:45725-45739.
27. Ballard L, Kamara S, Monrose F. Achieving efficient conjunctive keyword searches over encrypted data. In: Sihan Q, Wenbo M, Javier L, Guilin W, eds. *International Conference on Information and Communications Security*. Springer; 2005:414-426.

28. Poon HT, Miri A. An efficient conjunctive keyword and phase search scheme for encrypted cloud storage systems. In: *2015 IEEE 8th International Conference on Cloud Computing*. IEEE; 2015:508-515. doi:10.1109/CLOUD.2015.74
29. Sun W, Liu X, Lou W, Hou YT, Li H. Catch you if you lie to me: efficient verifiable conjunctive keyword search over large dynamic encrypted cloud data. In: *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE; 2015:2110-2118. doi:10.1109/INFOCOM.2015.7218596
30. Zhang L, Zhang Y, Ma H. Privacy-preserving and dynamic multi-attribute conjunctive keyword search over encrypted cloud data. *IEEE Access*. 2018;6:34214-34225.
31. Blei DM, Ng AY, Jordan MI. Latent Dirichlet allocation. *J Mach Learn Res*. 2003;3(Jan):993-1022.
32. Boneh D, Waters B. Conjunctive, subset, and range queries on encrypted data. In: Vadhan SP, ed. *Theory of Cryptography Conference*. Springer; 2007:535-554.
33. Zhang B, Zhang F. An efficient public key encryption with conjunctive-subset keywords search. *J Network Comput Appl*. 2011;34(1):262-267.
34. Xia Z, Zhu Y, Sun X, Chen L. Secure semantic expansion based search over encrypted cloud data supporting similarity ranking. *J Cloud Comput*. 2014;3(1):1-11.
35. Fu Z, Sun X, Linge N, Zhou L. Achieving effective cloud search services: multi-keyword ranked search over encrypted cloud data supporting synonym query. *IEEE Trans Consum Electron*. 2014;60(1):164-172.
36. Fu Z, Huang F, Ren K, Weng J, Wang C. Privacy-preserving smart semantic search based on conceptual graphs over encrypted outsourced data. *IEEE Trans Inf Forensics Secur*. 2017;12(8):1874-1884.
37. Fu Z, Xia L, Sun X, Liu AX, Xie G. Semantic-aware searching over encrypted data for cloud computing. *IEEE Trans Inf Forensics Secur*. 2018;13(9):2359-2371.
38. Dai H, Dai X, Yi X, Yang G, Huang H. Semantic-aware multi-keyword ranked search scheme over encrypted cloud data. *J Network Comput Appl*. 2019;147(1):102442.
39. Yang W, Zhu Y. A verifiable semantic searching scheme by optimal matching over encrypted data in public cloud. *IEEE Trans Inf Forensics Secur*. 2020;16:100-115.
40. Lang B, Wang J, Li M, Liu Y. Semantic-based compound keyword search over encrypted cloud data. *IEEE Trans Serv Comput*. 2021;14(3):850-863.
41. Scheuermann P, Ouksel M. Multidimensional B-trees for associative searching in database systems. *Inf Syst*. 1982;7(2):123-137.
42. Wang P, Ravishankar CV. On masking topical intent in keyword search. In: *2014 IEEE 30th International Conference on Data Engineering*. IEEE; 2014:256-267. doi:10.1109/ICDE.2014.6816656
43. Kermanshahi SK, Sun SF, Liu JK, et al. Geometric range search on encrypted data with forward/backward security. *IEEE Trans Dependable Secure Comput*. 2022;19(1):698-716.
44. Lang K. Newsweeder: learning to filter netnews. In: Armand P, Stuart R, eds. *Machine Learning Proceedings of the 1995*. Elsevier; 1995:331-339.
45. Gabryel M, Damaševičius R, Przybyszewski K. Application of the bag-of-words algorithm in classification the quality of sales leads. In: Leszek R, Rafał S, Marcin K, Witold P, Ryszard T, Jacek MZ, eds. *International Conference on Artificial Intelligence and Soft Computing*. Springer; 2018:615-622.

How to cite this article: Zhou Q, Dai H, Hu Z, Liu Y, Yang G. Accuracy-first and efficiency-first privacy-preserving semantic-aware ranked searches in the cloud. *Int J Intell Syst*. 2022;37:9213-9244. doi:10.1002/int.22989